

The ERP landscape, and exactly what to do next

Where the system stands, why nobody yet sees the same data, and the six steps that fix it — ending with a link and a password anyone can use. No Cloudflare, no domain, no laptop.

The one sentence that explains everything

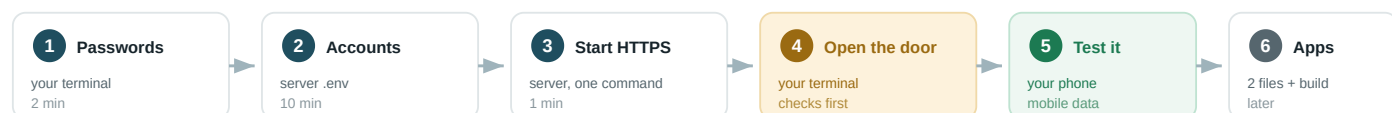
Your server works. The app on your phone has never been connected to it — it loads a static website that saves everything into that phone's own storage. Six steps connect them, and hand you a link and a password anyone can use.

Status at a glance

Part of the system	State	What that means
Server — Hetzner Cloud, 178.105.10.156	RUNNING	Application, database and nightly backups all up.
Database — PostgreSQL	VERIFIED	Customers written from the app land here and read back.
Deploying code — GitHub → server	WORKING	Both of you can ship. Tests now gate the release.
Login — named accounts for your team	BUILT	Tested end to end. Needs switching on (steps 1–4).
Shared access — one link, one password	BUILT	Anyone you send it to signs in with no account. Live data.
Reachable from a phone	NOT YET	Needs steps 3–5. Everything for it is written.
Phone apps — iOS & Android	WRONG TARGET	Point at a static website, not your server. Step 6.
Shared calendar screen	NOT SHARED	Data and commands work; the screen still saves locally.
Limits on what a visitor may do	NONE	Shared access can change and delete live records. See page 6.
Restore from backup	UNTESTED	Backups are taken. Restoring has never been rehearsed.

Read this before the steps. Steps 1–5 are done once, in about half an hour. Step 4 is the only one that exposes the server, and it refuses to run until the login genuinely works — the order matters, and the system enforces it.

The six steps, at a glance

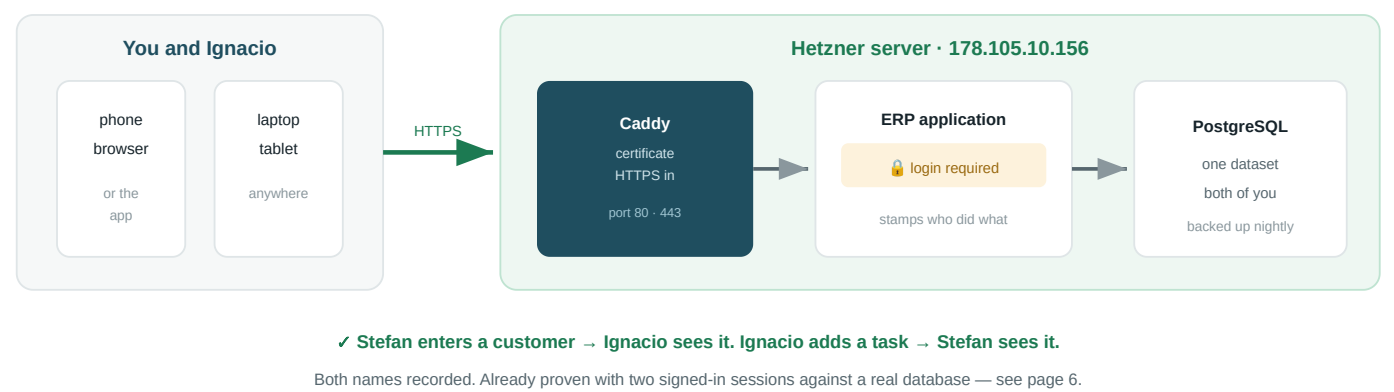


Today: three separate islands

Data does not move between these. That is the whole problem.



After the six steps: one system



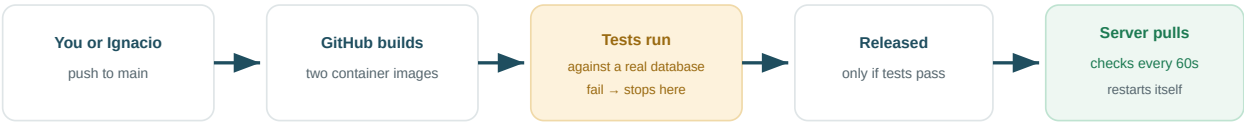
Where each kind of data actually lives

What you enter	Today	After the six steps
Customers & suppliers	on that device in the phone app	shared — already proven working
Money in and out	on that device	shared
Site progress, change orders	on that device	shared
Calendar tasks	on that device	half — the data is shared, the calendar screen still saves locally until it is moved
Quotes, contracts, invoices	on that device	not yet — the server does not accept these operations yet

The two amber rows are the honest remaining gap, and they are separate from anything in this document's six steps. Neither blocks the pilot: customers, money and site progress are the parts you need shared first.

Deploying code — this already works, for both of you

Neither of you needs a key, a password or a laptop to ship a change. You push; the server collects it. Nothing ever logs into the machine.



Total: roughly four minutes from pushing to running. Only the main branch releases — a feature branch is built and tested but not shipped.

Fixed this week: the release used to happen BEFORE the tests finished. It no longer does.

Running the server itself — from a browser, no laptop

Health checks and backups now run from a button. One-time setup, done on your phone if you like.

Where	What
GitHub → Settings → Secrets and variables → Actions	Add two secrets, once: <code>HCLOUD_TOKEN</code> (your Hetzner API token) and <code>SERVER_SSH_KEY</code> (the whole private key file from <code>ops/.provisioned/id_ed25519</code>).
GitHub → Settings → Environments → production	Add yourself as a required reviewer . Then every ops run waits for your approval — worth doing, because anyone with write access to the repository can otherwise use those secrets.
GitHub → Actions → “Ops (run from a browser)”	Run workflow → choose <code>status</code> , <code>backup-now</code> or <code>list-ssh-allowed</code> . Read the log: every line is a ✓, a ! or a ✗, and the ones that are not ✓ say what to do.

Two people, one server. The old script set the SSH allow-list to exactly one address, so whoever ran it second locked the first one out — silently. Use `./ops/ssh-allow.sh add` instead: it changes one entry and leaves everyone else alone. If you ever do lock yourself out, Hetzner's web console reaches the machine anyway: **console.hetzner.com** → **the server** → **Console**.

Who does what, once the pilot is on

Task	Who	Where
Use the ERP day to day	You and Ignacio	The address from step 5, any device, after signing in.
Ship a code change	You and Ignacio	Push to <code>main</code> . Tests gate it; the server collects it within a minute.
Check the server is healthy	You and Ignacio	GitHub → Actions → Ops → <code>status</code> . No key needed.
Add a colleague, or change the shared password	You	Edit <code>ERP_USERS</code> or <code>ERP_ACCESS_PASSWORD</code> in the server's <code>.env</code> , then restart.
Open or close internet access	You	<code>./ops/open-web.sh</code> or <code>--close</code> .
Backups, certificate renewal	Automatic	Nightly at 02:30 UTC, encrypted. Nothing to remember.

Keep the last two to yourself for now. Both mean editing the server's `.env`, which is also where the database passwords live — the difference between a colleague having an account and a colleague having the keys.

Turning the pilot on

Six steps, about half an hour. Steps 1–2 create the accounts, 3–4 put the ERP on the internet behind them, 5–6 get it onto the phones.

1 Make a password for each of you WHERE — a terminal, in your clone of the repository

```
node apps/web/scripts/hash-password.mjs stefan@caneisubirats.com
node apps/web/scripts/hash-password.mjs ignacio@caneisubirats.com
```

Asks twice without showing it, then prints one long line per person. Copy both. Nothing is sent anywhere. Use at least 12 characters from the password manager — the ERP will be on the internet and there is no lockout after repeated guesses yet.

2 Put the accounts on the server WHERE — on the server, in the file /opt/canei-erp/.env

`ssh -i ops/.provisioned/id_ed25519 root@178.105.10.156` then `nano /opt/canei-erp/.env` — or Hetzner's web console.

```
# the two lines you copied in step 1, comma-separated, all on one line
ERP_USERS="stefan@caneisubirats.com:scrypt$...,ignacio@caneisubirats.com:scrypt$..."

# run: head -c 48 /dev/urandom | base64 and paste the result here
SESSION_SECRET="..."

# THE ONE YOU HAND OUT. No account, no email — just this.
ERP_ACCESS_PASSWORD="Obras-2026-Barcelona"
ERP_DEFAULT_TENANT="diorka"

# your server's address with dots replaced by dashes
PUBLIC_HOSTNAME="178-105-10-156.sslip.io"
ACME_EMAIL="stefan@caneisubirats.com"
```

`ERP_ACCESS_PASSWORD` is what makes "a link and a password" work: whoever has it signs in by leaving the email box **empty**. Named accounts put a real name in the audit trail; a shared session is recorded as `invitado-XXXX`. `SESSION_SECRET` must be set alongside them — without it the application refuses to start rather than recording everyone's work under one name.

3 Start the piece that handles HTTPS WHERE — on the server, still in that SSH session

```
cd /opt/canei-erp
docker compose -f docker-compose.prod.yml --profile pilot up -d
```

Starts Caddy, which obtains a real certificate for `178-105-10-156.sslip.io` — a free naming service turning your IP into a proper name. It matters because iPhones refuse a self-signed certificate outright. When your domain transfers you change `PUBLIC_HOSTNAME`, nothing else.

Do not skip ahead to step 4. It opens the server to the world, and checks first — on the machine — that a request with no login is actually turned away, not merely that you configured one. If the running application predates the login it refuses and opens nothing.

Opening it up, and getting it on the phones

4 Open the door WHERE — a terminal, in your clone (needs ops/provision.conf)

```
./ops/open-web.sh
```

Opens ports 80 and 443 — but only after confirming the login is real. Wait a minute: the certificate is obtained on the first request.

To take it back off the internet at any time: `./ops/open-web.sh --close`. SSH is unaffected and nothing is lost — the address simply stops answering.

5 Check it from your phone — on mobile data WHERE — the browser on your phone, wifi turned off

```
https://178-105-10-156.sslip.io
```

Wifi off, so you are testing the real path in. You should get the Canei sign-in screen. Try **both** ways: your own address and password, then the shared password with the email box left empty — that second one is what you will be sending out.

What to send somebody:

```
Aquí tiene el acceso al sistema:
https://178-105-10-156.sslip.io
Contraseña: Obras-2026-Barcelona
Deje el campo de correo vacío. Funciona en portátil, tablet y móvil.
```

6 Point the phone apps at the server WHERE — two files in the repository, then a new build

```
ios/CaneiSubirats/Support/Config.swift — line 20
static let baseUrl = URL(string: "https://178-105-10-156.sslip.io/")!

android/app/src/main/kotlin/com/caneisubirats/app/MainActivity.kt — line 26
private val baseUrl = "https://178-105-10-156.sslip.io/"
```

Both currently load `stefangruber001.github.io`, which is the static demo site with no server behind it — the direct cause of the calendar not being shared. After changing these, a new TestFlight and Play build is needed. **Until then, use the address from step 5 in the phone's browser** — it is exactly the same application.

The ten-second test, whenever something looks unshared

What the address bar says	What that means
<code>stefangruber001.github.io/...</code>	LOCAL ONLY Saved on that device. Nobody else will ever see it, however convincingly it appears to save.
<code>178-105-10-156.sslip.io</code>	THE SERVER Shared, backed up, attributed to you.

If two people are looking at two different addresses, they are looking at two different datasets. This is the single rule that explains the calendar problem, and it will explain the next one too.

What you will and will not have

✓ Works after these steps

- **Anyone you send the link and password to** signs in from any device — laptop, tablet, iPhone — with no account to create.
- Your own team sign in by name; the audit trail says so.
- One shared dataset. A customer entered by one is visible to the other immediately.
- Every change records **who** made it — verified with two separate sessions writing to a real database.
- Simultaneous edits are safe: the second save is refused with “somebody saved before you”, never a silent overwrite.
- Nightly encrypted backups continue; you can ship code without touching the server.

✗ Still missing

- **A full project from lead to close.** The server accepts twelve operations — customers, money, progress, change orders, tasks, the quarterly archive. Not quotes, contracts or invoices.
- **A shared calendar screen.** The data and operations work; the screen itself still saves into the browser.
- **A rehearsed restore.** Backups are taken; restoring has never been tried.
- **Any limit on what a visitor may do.** The link and password do everything your own team can, including deleting records. Read-only access does not exist yet.
- **Password reset, logout, two-factor.** Resetting one means editing `.env`.

Four risks worth knowing about

Risk	What it means, and what to do
The shared password reaches the live books	Deliberate — you are showing the real system, not a sample. Anyone you send the link to can change or delete real records and will see real customer names and figures. Two habits: before a session take a backup (Actions → Ops → <code>backup-now</code>); after one change <code>ERP_ACCESS_PASSWORD</code> . Worth deciding first: those names are third parties' personal data, and showing them is a disclosure.
No limit on password guessing	Nothing yet slows an automated attempt at the login form, and a shared password gets typed in front of people and written down — make it a long phrase. Rate limiting is a small, known piece of work.
The address depends on a free service	<code>sslip.io</code> turns your IP into a name. If it stopped, the address would stop resolving until it returned. No data is at risk — it is on your server and in your backups. Moving to <code>caneisubirats.com</code> removes this entirely.
A lost phone stays signed in	Sessions last eight hours and cannot be cancelled individually. Changing <code>ERP_ACCESS_PASSWORD</code> stops new sign-ins; changing <code>SESSION_SECRET</code> too ends open ones immediately, yours included.

What was actually tested, not assumed

Driven against a real build, a real PostgreSQL and the restricted database account: an anonymous browser is redirected to sign in and an anonymous API call refused; a visitor signed in with **only the shared password and no email** and worked on the live company; a near-miss password was refused; staff signed in by name and saw the visitor's work, the database recording `invitado-d31d → addParty`.

Those runs also found two genuine bugs: adding a task discarded the person's name; and a server reached only through the shared link would have had its lock switched off entirely. Both fixed, with tests.

Full written detail in the repository: `docs/PILOT-WITHOUT-CLOUDFLARE.md` (these steps), `docs/OPS-WITHOUT-A-LAPTOP.md` (running the server from a browser), `docs/WHY-THE-CALENDAR-IS-NOT-SHARED.md` (the calendar).